

Aperture

Natural-language SQL analyst — study guide for demo & interview

What the project is. Ask a question in English; Aperture retrieves the relevant subschema, writes SQL, validates it against a parsed syntax tree, executes it read-only, reads any database error and repairs itself in a loop, then returns an answer plus a validated chart spec. Built on LangGraph, LangChain and LangSmith, with models served from AWS Bedrock.

Guardrails, three independent layers: a Postgres role that physically cannot write, an AST validator that rejects non- `SELECT` statements, and a planner cost check that refuses queries before they scan the table.

Tier 1 — you will be asked about these **CORE**

LangGraph

- State machine over a `TypedDict`: nodes mutate state, edges route between them.
- **Conditional edges and cycles** — the whole reason this isn't a chain. `validate → execute → (error) → diagnose → generate` is a loop; LCEL chains are DAGs and cannot express it.
- Checkpointers give conversation memory and crash resume. Aperture uses the SQLite checkpointer so the tool carries its own state instead of demanding write access to the database it analyses.
- `interrupt()` pauses the graph mid-run for human approval and resumes from the checkpoint.
- `astream_events` is the event stream that feeds server-sent events to the UI.

"Why not just a chain?" Retry-until-valid is a cycle over mutable state. A chain would push that loop into plain Python outside the framework, losing checkpointing, tracing and resume.

sqlglot

- Parses SQL into an expression tree: `parse`, `parse_one`, `exp.Select`, `find_all`, `.sql(dialect=...)`.
- Dialect transpilation — one AST renders as Postgres, MySQL or SQLite.
- **Why an AST beats a regex:** a comment before `DROP TABLE`, a `DELETE` hidden inside a CTE, a second statement after `;`. A regex misses all three; a parser cannot.
- Read `guards/validator.py` — roughly 120 lines, and it is the answer to "how do you stop the model dropping a table".

Postgres catalog and planner

- `pg_class`, `pg_attribute`, `pg_constraint`, `pg_enum` — how 56 tables are read in three queries instead of 56 round-trips.
- `pg_stats`: `most_common_vals`, `n_distinct`, `null_frac`. The planner already computed these, so profiling borrows them and costs nothing. The cleverest part of the project — know it cold.

- `EXPLAIN (FORMAT JSON)` → `Total Cost` , `Plan Rows` . Cost is in arbitrary planner units, *not* milliseconds. Be precise about that.
- Least privilege: `GRANT SELECT` , `default_transaction_read_only` , `statement_timeout` .

Text-to-SQL as a field

- **Schema linking** — choosing the right subschema. **Value linking** — choosing the right literals.
- Value linking is the dominant failure mode. The demo database proves it: the status column contains `CONFUSED_CUSTOMER` , a literal no model would ever guess.
- **Execution accuracy** versus exact string match: compare result sets, order-insensitively, because many different SQL strings are equally correct.
- BIRD versus Spider: BIRD is harder, dirtier, and ships external-knowledge hints. Benchmark contamination is a fair criticism — models have seen Spider in training.

AWS Bedrock

- The Converse API gives one interface across model families. Auth is SigV4 through boto3's credential chain — on this machine the keys live in `~/.aws/credentials` , not in environment variables.
- Mantle is Bedrock's inference engine, exposing OpenAI-compatible surfaces (`/responses` , `/chat/completions` , `/messages`) at `bedrock-mantle.{region}.api.aws` .
- Model ids are region-scoped: `qwen.qwen3-coder-next` resolves in `us-east-1` and returns an invalid-identifier error in `ap-south-1` . A good anecdote — it was hit live during setup.

Say this carefully. Per-token pricing for that model is unverified here; the spend ledger uses configured, deliberately pessimistic rates so the ceiling trips early rather than late. Describe the cost as "estimated and configurable" — never quote a price as fact.

Tier 2 — know the shape, not every detail SUPPORTING

Embeddings and retrieval

- BGE-base-en-v1.5, 768 dimensions, cosine similarity. Vectors are normalised, so cosine reduces to a dot product.
- **Two-tier retrieval:** embed one document per table first; embed columns only inside candidate tables afterwards. This is the answer to "what if the database is huge" — a 50,000-table warehouse never embeds a million columns.
- Why no vector database by default: under a thousand vectors, a brute-force NumPy scan takes about 0.3 ms. Know roughly where that breaks (tens of thousands of vectors, bounded by RAM) and name HNSW as what a real index does instead.

networkx and graph retrieval

- BFS expansion, shortest path, junction tables, fan-out detection.
- **The strongest technical story in the project:** the status-history table holds 16,812 rows against the orders table's 2,999. A perfectly legal foreign-key join inflates any `SUM` over order totals by roughly 5.6×, and raises no error at all — just a wrong number. Structure tells you the join is legal; only data tells you it lies.

LangSmith

- Traces are nested runs, one per graph node. Datasets plus evaluators form the eval harness. Feedback captures thumbs up/down for later triage.
- Traces carry query results off the machine — worth stating plainly, since the demo database holds real customer rows.

SQLAlchemy 2.0 and psycopg3

- `create_engine`, `text()`, connections versus sessions.
- Why `Inspector` is avoided at scale: it issues one round-trip per table.

Vega-Lite

- Grammar of graphics: `mark` plus `encoding` (x, y, colour channels) and field types — `quantitative`, `temporal`, `nominal`.
- **Why a declarative spec rather than generated plotting code:** a spec is validated JSON with no execution. An agent that emits Python to draw charts is a remote-code-execution hole wearing a nice hat.

Tier 3 — skim SURFACE

- FastAPI async, server-sent events versus WebSockets, the browser `EventSource` API.
- React with Vite and TypeScript, `react-vega`, shadcn/ui components.
- MCP: servers, tool definitions, stdio transport — and why it makes the database reachable from any agent or IDE.
- `uv`, `pyproject.toml`, console-script entry points.

If you have two hours, study in this order

1. LangGraph cycles, checkpointers, interrupts.
2. `guards/validator.py`, and why an AST beats a regex.
3. `pg_stats`, and what profiling buys that DDL cannot.
4. Schema linking versus value linking.
5. Execution accuracy — and why no number can be cited without ground truth.

The six questions judges actually ask

1. How do you stop it dropping a table?

Three independent layers: a database-enforced read-only role, so `DELETE` fails at the server with *cannot execute DELETE in a read-only transaction*; an AST validator that rejects anything that is not a single read statement; and a statement timeout. Demo the error live — it lands better than any slide.

2. How is this different from asking ChatGPT for SQL?

It executes the query, reads the real database error, and repairs itself. It also cannot write, cannot scan an entire table, and cannot invent an enum value it has never seen.

3. Does it scale past a toy database?

Introspection cost is flat in table count, retrieval is two-tier, and profiling borrows planner statistics instead of scanning.

4. How do you know the answers are right?

Execution accuracy against a benchmark with gold SQL. Be straight that the live demo database has no ground truth, so no accuracy number is claimed for it.

5. Why LangGraph rather than a plain chain?

Cycles, checkpoints and interrupts — each load-bearing, none decorative.

6. What breaks it?

Ambiguous business terms, fan-out joins, and empty tables. Naming your own failure modes reads as competence, not weakness.

Build status at time of writing

On disk: `config.py`, `budget.py` (token and spend ledger with a hard ceiling), `llm.py`, `guards/validator.py`, `guards/cost.py`, `db/connection.py`, `db/introspect.py`, `db/profile.py`, plus repository scaffolding and an MIT licence. The `aperture_ro` Postgres role exists and is verified: reads succeed, writes are refused by the server. Not yet written: the foreign-key graph, embeddings and linker, the LangGraph loop, the chart node, the SSE server, the CLI, the MCP server, the React client, and the benchmark harness. Nothing is committed yet.